

recall_telemetry_probes

recall_telemetry_probes.sh — companion guide

Revision: 1 Last modified: 2026-06-10T11:40:36Z

Overview

tests/security/recall_telemetry_probes.sh is a black-box, anti-bluff security probe suite for two Helix OTA control-plane surfaces that the sibling suites (security_probes.sh, security_probes_filters.sh) do **not** cover:

- POST /api/v1/deployments/{id}/recall — server-driven rollback control.
- GET /api/v1/devices/{id}/telemetry — ownership + filter input validation.

It boots a **real** ota-server over real HTTP and HARD-asserts the **actual** status code returned by a real response for every probe (Constitution §11.4 / §11.4.27 / §11.4.123). A genuine defect FAILs; nothing is papered over. The suite is self-driving and self-cleaning (server stopped + workdir removed on every exit path via trap ... EXIT INT TERM, §11.4.14).

Status codes are read from source, not assumed (§11.4.6). Both handlers use respondValidation(), which returns **HTTP 400** (CodeValidationFailed) — **not 422**. The probe expectations below reflect the real handler behaviour, confirmed against server/internal/api/errors.go, handlers_recall.go, and handlers_telemetry.go.

Prerequisites

- Required: bash, curl, jq, go (self-host build of cmd/ota-server).
- Optional (only the recall-VALIDATION fixture V1/V2 needs them): openssl (≥3, ed25519), xxd, base64, python3. When absent, V1/V2 SKIP-with-reason (§11.4.3) — never a false PASS — and every other probe still runs.

Usage examples

Self-hosted (recommended; default — ephemeral per-PID port in 20000–39999):

```
bash tests/security/recall_telemetry_probes.sh
```

Pin a port / point the evidence file:

```
HELIX_PORT=28765 \
HELIX_SECURITY_EVIDENCE=docs/qa/<run-id>/
  RUN_EVIDENCE_RECALL_TELEMETRY.txt \
  bash tests/security/recall_telemetry_probes.sh --port 28765
```

Against an already-running server (V1/V2 will SKIP unless a key-configured server with a seeded deployment exists):

```
bash tests/security/recall_telemetry_probes.sh \
  --external --base-url http://127.0.0.1:8080 --password
  "$HELIX_ADMIN_PASSWORD"
```

Exit codes: 0 PASS, 1 FAIL (≥ 1 hard assertion failed), 3 ABORT (missing tool / build / unhealthy server).

Probes and EXACT expected status

Probe	Request	Expected
R1	unauth POST /deployments/{id}/recall	401
R2	device-role token POST .../recall	403
R2b	device token GET /client/update (token-valid control)	200/204
R3	operator/admin token POST .../recall (valid target, real fixture)	201 (SKIP if no signed fixture)
V1	POST .../recall missing to_release_id (deployment has a release)	400 (SKIP if no fixture)
V2	POST .../recall unknown to_release_id (deployment exists)	404 (SKIP if no fixture)
V3	POST .../recall unknown deployment id (operator token)	404
O1	device A token GET /devices/{B}/telemetry	403
O2	device A token GET /devices/{A}/telemetry (own)	200
O3	privileged (admin/viewer) token GET /devices/{B}/telemetry	200
F1	?event=<unknown>	400
F2	?since=<not-RFC3339>	400
F3a/b/c	?limit=0 / =999 / =-1 (out of [1,200])	400 each
F4	?cursor=<malformed>	400
F5		200

Probe	Request	Expected
	? event=success&limit=10&cursor=0 (valid control)	
T1	POST .../recall with injected public_key/signature fields	201 (valid target + fixture) / 404 (no fixture) — fields IGNORED, no request key path
T1b	T1 did not 5xx	no 5xx
*b	injection/edge inputs did not 5xx	no 5xx

Why the codes are what they are

- **Authz** is enforced by `requireRole (middleware.go)` BEFORE the handler: unauth → 401, wrong role → 403, regardless of whether the deployment exists. recall is wired `requireRole(RoleOperator, RoleAdmin)`; telemetry is `requireRole(RoleViewer, RoleOperator, RoleAdmin, RoleDevice)`.
- **Recall validation** (`handleRecall`): unknown deployment → 404; malformed body / missing `to_release_id` → 400 (`respondValidation`); unknown `to_release_id` → 404; success → 201.
- **Telemetry ownership** (`handleDeviceTelemetry`): a device-role token may read only its own `deviceId` (`claims.Subject != deviceId` → 403); a privileged token reads any.
- **Telemetry filter validation**: event (closed set), since/until (RFC3339), limit ([1,200]), cursor (`int ≥ 0`) all reject via `respondValidation` → 400.
- **Trust boundary**: `RecallRequest` binds ONLY `{to_release_id, reason}`. `handlers_recall.go` has **no** `resolvePublicKey/signature` path, so extra request fields (`public_key, signature, ...`) are silently ignored by the JSON binder and can never inject a trusted key.

Token-class limitation (documented, not bluffed)

The self-hosted `cmd/ota-server` static user directory mints exactly one login user carrying `{admin, operator, viewer}` combined — there is no env knob for a **viewer-only** user. The genuinely non-privileged subject available to drive the recall 403 is therefore the **device-role** token (R2), which is the correct and provable rejection. R3 proves the operator/admin path is allowed. A dedicated *viewer-only-rejected-on-recall* probe is not assertable in self-host mode without a runtime user-creation surface; this is a known coverage edge, not a faked PASS.

Internal behaviour

1. Optionally generate an ephemeral ed25519 keypair (never committed; lives in a `mktemp` dir removed on exit, §11.4.10).
2. `go build ./cmd/ota-server`, boot it with ephemeral admin/token secrets (and `HELIX_ARTIFACT_PUBKEY` when signing is available), wait for `/healthz`.
3. Admin login; register two devices for the ownership probes.

4. If signing is available, mint a signed base+target artifact → two releases → one deployment (the recall-validation fixture; reuses the tests/e2e/pipeline_signed.sh recipe).
5. Run R/V/O/F/T probes, each tee'd (status + body excerpt) to the evidence file.
6. trap cleanup EXIT INT TERM kills the server and removes the workdir.

curl runs with -g (globbing off) so [] \$ in query params are sent literally instead of erroring into a 000 script-bug FAIL-bluff (§11.4.1).

Related scripts

- tests/security/security_probes.sh — base black-box security suite (same harness).
- tests/security/security_probes_filters.sh — filter/pagination security suite.
- tests/e2e/pipeline_signed.sh — signed-artifact pipeline (fixture recipe source).

Cross-references (server source)

- server/internal/api/handlers_recall.go — handleRecall (404/400/201, no key path).
- server/internal/api/handlers_telemetry.go — handleDeviceTelemetry (ownership 403 + filter 400s).
- server/internal/api/server.go — route role wiring.
- server/internal/api/errors.go — respondValidation == HTTP 400.
- server/internal/api/middleware.go — requireRole (401/403).

Last verified date: 2026-06-10 (parse-only: bash -n + sh -n clean; live run deferred to the conductor against the clean merged tree).